Deep Learning for Encrypted Network Traffic Classification and Threat Detection

Avula Praveen Kumar
Student ID: 24271730
H9AIMLC – AI/ML in Cybersecurity
National College of Ireland
Dublin, Ireland

***Abstract*—The proliferation of encryption protocols such as TLS 1.3 and HTTPS has rendered traditional Deep Packet Inspection (DPI) methods ineffective for network traffic analysis, creating significant blind spots for Security Operations Centres. This study proposes a hybrid CNN-LSTM architecture with temporal attention for classifying encrypted network traffic without decryption, using flow-level statistical features. Evaluated on the ISCX-VPN-NonVPN benchmark dataset containing 28 statistical features across 7 traffic categories, the proposed model is compared against standalone CNN, LSTM, and Random Forest baselines. Results demonstrate that the CNN-LSTM-Attention hybrid achieves a macro-F1 of 0.835 and accuracy of 0.834, while all deep learning models maintain sub-0.12 ms inference latency suitable for real-time deployment on CPU hardware. Notably, Random Forest attains the highest overall F1 of 0.895, highlighting the continued strength of traditional methods on structured tabular features. The attention mechanism further provides an interpretability contribution by revealing which temporal feature segments are most salient for classification.**

***Index Terms*—Encrypted Traffic Classification, Deep Learning, CNN-LSTM, Attention Mechanism, Network Security**

I. Introduction

The expansion of encryption protocols including TLS 1.3, HTTPS, and end-to-end encryption has fundamentally changed network security monitoring. While encryption protects user privacy and data integrity, it creates significant coverage gaps for traditional security monitoring systems that depend upon Deep Packet Inspection to analyse packet contents. This challenge affects the detection of malware Command & Control communications, data exfiltration, and encrypted ransomware traffic without compromising encryption-based privacy protections.

Deep learning methods, particularly CNNs and LSTMs, have shown promise for automatic application identification in encrypted traffic without decryption. These models learn hierarchical representations from packet-level metadata features—inter-arrival times, sizes, flow duration, and flag statistics—that produce unique fingerprints for different application types [1].

This study addresses the research question: *How can a hybrid deep learning architecture effectively classify encrypted network traffic and detect malicious communications, achieving macro-F1 $\geq$ 0.85 with inference latency $<$ 1 ms, while maintaining computational efficiency suitable for real-time deployment?*

The ISCX-VPN-NonVPN dataset is employed, containing over 150,000 labelled bidirectional flows with 28 statistical features from both VPN-tunnelled and regular network traffic.

II. Related Work

A. Foundational Techniques and Classical Approaches

The attention mechanism, first introduced by Bahdanau et al. [2] for neural machine translation, enabled models to selectively focus on relevant input segments rather than compressing entire sequences into fixed-length representations. This concept was subsequently generalised by Vaswani et al. [3] in the Transformer architecture, which relies entirely on self-attention and has since become the dominant paradigm across natural language processing and, increasingly, network traffic analysis. Our work adopts the additive attention formulation of Bahdanau et al. rather than multi-head self-attention, as the lower computational overhead is better suited to real-time inference on CPU hardware.

Prior to the deep learning era, network traffic classification relied on port-based heuristics and statistical machine learning. Moore and Zuev [4] pioneered the use of Bayesian analysis on hand-crafted flow statistics, demonstrating that statistical features such as packet sizes and inter-arrival times carry discriminative information even without payload inspection. While effective for unencrypted traffic, their Naïve Bayes approach assumed feature independence—an assumption violated by the correlated nature of network flow statistics—and could not scale to the diverse encrypted application landscape that exists today.

Pacheco et al. [5] provided a comprehensive survey of machine learning solutions for network traffic classification, cataloguing feature engineering strategies, dataset characteristics, and deployment considerations. Their survey identified a persistent gap between research accuracy figures and production deployment feasibility, a finding that directly motivates the latency-aware evaluation methodology adopted in the present study.

B. Deep Learning for Encrypted Traffic

The application of CNNs to raw network traffic was pioneered by Wang et al. [6], who treated packet payloads as 1D signals and demonstrated that convolutional filters can automatically learn discriminative byte-level patterns. Their

approach eliminated the need for manual feature engineering but required access to raw payload bytes, which is infeasible when traffic is encrypted end-to-end. In contrast, our CNN component operates on pre-extracted statistical features, preserving privacy while retaining the spatial pattern extraction capability.

Lotfollahi et al. [7] proposed Deep Packet, combining a stacked autoencoder with a CNN for encrypted traffic classification, achieving over 98% recall on ISCX data. However, their architecture processes raw packet bytes and requires substantially more training data to converge. Their work nonetheless demonstrated that deep representations can outperform hand-crafted features when sufficient raw data is available, suggesting a promising direction for future extensions of our approach.

The LSTM architecture, introduced by Hochreiter and Schmidhuber [8], addresses the vanishing gradient problem in recurrent networks and has been widely adopted for sequential network traffic modelling. Its gating mechanism—input, forget, and output gates—enables the network to selectively retain or discard temporal information across long sequences. Our hybrid architecture leverages bidirectional LSTMs to capture both forward and backward temporal dependencies in flow feature sequences, an approach that has not been extensively explored in the encrypted traffic classification literature.

Rezaei and Liu [9] provided an overview of deep learning methods for encrypted traffic classification, identifying key challenges including dataset imbalance, concept drift, and the trade-off between classification accuracy and computational cost. Their analysis concluded that hybrid architectures combining spatial and temporal feature extractors represent a promising research direction, a finding that directly supports the CNN-LSTM design adopted in this study.

C. Website and VPN/Tor Fingerprinting

The ISCX-VPN-NonVPN dataset used in this study was introduced by Draper-Gil et al. [10], who characterised encrypted and VPN traffic using time-related features and demonstrated that flow-level statistics can distinguish VPN-tunnelled traffic from regular encrypted sessions. Their feature engineering methodology—based on CICFlowMeter—established the 28 statistical features that form the input space for all models evaluated in this study.

Panchenko et al. [11] addressed website fingerprinting at Internet scale, showing that traffic analysis attacks can identify visited websites even through Tor and VPN tunnels by exploiting packet size and timing patterns. Their work raised significant privacy concerns about the feasibility of traffic analysis on anonymised connections, a consideration directly relevant to the ethical implications of the classification system proposed here.

Hayes and Danezis [12] proposed k-fingerprinting, a scalable website fingerprinting technique using random forests on hand-crafted traffic features. Their method achieved high accuracy in both closed-world and open-world settings, demonstrating that ensemble tree methods are particularly effective on structured traffic features—a finding consistent with the strong Random Forest baseline performance observed in our experiments.

Sirinam et al. [13] applied deep CNNs to website fingerprinting, achieving state-of-the-art accuracy that surpassed traditional feature-based approaches even against traffic obfuscation defences. Their Deep Fingerprinting model demonstrated that sufficiently deep neural networks can learn representations robust to padding and timing perturbations, though at the cost of significantly increased computational requirements compared to the lightweight architecture proposed in this work.

D. Recent Advances

Liu et al. [1] proposed ATVITSC combining Vision Transformer with spatial-temporal feature extraction, achieving macro-F1 scores exceeding 97% across VPN, Tor, malware, and mobile traffic. However, the approach requires pre-generated session-level images and is computationally intensive.

Zhu et al. [14] proposed BCFNet combining ET-BERT global features with Inception 1D-CNN local features, achieving 98.88% accuracy on the ISCX VPN-nonVPN dataset. However, reliance on pre-trained ET-BERT requires significant computational resources.

Luo et al. [15] proposed the ICE-CP framework addressing concept drift in malicious traffic identification using variational autoencoders. However, the focus is primarily on malicious traffic without comprehensive multi-class application classification.

Wang et al. [16] combined Gated Dilated Convolution for spatial features with CA-LSTM for temporal features, achieving over 95% accuracy on ISCX VPN-NonVPN. However, increased model complexity limits real-time deployment capability.

Liu et al. [17] combined ECA-Net modules with CNN and Transformer encoders, achieving 98.25% accuracy with faster convergence. However, Transformer memory requirements limit interpretability and deployment on resource-constrained hardware.

Chen, Wei and Wang [18] created graph representations of packet-level traffic with lightweight weighted output matrices. However, GNN inference latency may exceed requirements for real-time applications.

Elshewey and Osman [19] benchmarked multiple deep learning architectures for HTTPS traffic classification, with their ensemble achieving 99.49% accuracy. However, ensemble inference complexity increases latency compared to single-model approaches.

III. Methodology and Implementation

A. Dataset and Preprocessing

The ISCX-VPN-NonVPN dataset provides 28 flow-level statistical features including flow duration, packet counts, byte volumes, packet length statistics, inter-arrival times, and TCP

flag counts. Preprocessing includes StandardScaler normalisation and stratified 70%/15%/15% train/validation/test splitting.

B. Model Architectures

CNN Classifier: Two 1D convolutional layers with 64 and 128 filters (kernel size 3, stride 1, padding 1) extract spatial patterns from flow features, each followed by batch normalisation and ReLU activation. Adaptive average pooling reduces the temporal dimension to 1 before a fully connected classification head with a hidden layer of 128 units. Dropout of 0.3 is applied before the output layer.

LSTM Classifier: A two-layer LSTM with 64 hidden units per layer and dropout of 0.3 between layers captures temporal dependencies across the feature sequence. The final hidden state is passed through a fully connected layer of 128 units with ReLU activation and dropout of 0.3 before the softmax output layer.

CNN-LSTM with Attention (Proposed): The hybrid architecture processes flow features through the same two-layer 1D CNN block (64 and 128 filters, kernel size 3) for spatial pattern extraction. The CNN output is then fed into a single-layer bidirectional LSTM with 64 hidden units per direction (128 total), producing a sequence of hidden states $\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_T$. A temporal attention mechanism computes scalar attention energies for each time step:

$$e_t = \mathbf{v}^T \tanh(\mathbf{W}\,\mathbf{h}_t + \mathbf{b}), \tag{1}$$

where $\mathbf{W} \in \mathbb{R}^{d_a \times 2d_h}$ is the attention weight matrix, $\mathbf{b} \in \mathbb{R}^{d_a}$ is the bias, $\mathbf{v} \in \mathbb{R}^{d_a}$ is the context vector, and $d_a = 64$ is the attention dimensionality. The attention weights are then obtained via a softmax normalisation:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^{T} \exp(e_j)}. \tag{2}$$

The context vector $\mathbf{c} = \sum_{t=1}^{T} \alpha_t\,\mathbf{h}_t$ forms a weighted summary of the LSTM output sequence, which is passed through a fully connected layer of 128 units with ReLU and dropout of 0.3 before the final classification layer.

Random Forest Baseline: 200 estimators with max depth 20, minimum samples split of 2, and minimum samples leaf of 1, providing a traditional ML comparison point that operates directly on the tabular feature space.

C. Training Configuration

All deep learning models are trained using the Adam optimiser with an initial learning rate of 1×10^{-3}, weight decay of 1×10^{-4}, and a ReduceLROnPlateau scheduler that reduces the learning rate by a factor of 0.5 when validation loss plateaus for 5 consecutive epochs (patience = 5). Training runs for a maximum of 50 epochs with a batch size of 256, and early stopping with a patience of 10 epochs is employed to prevent overfitting. Cross-entropy loss is used as the training objective for all neural network models.

To ensure statistical robustness, all experiments are conducted across 5 independent runs with different random seeds (seeds 0, 1, 2, 3, 4), and we report the mean $\pm$ standard deviation of each metric. Additionally, 5-fold stratified cross-validation is performed on the training partition to verify that performance estimates are stable across data splits. For Random Forest, scikit-learn's `random_state` parameter is varied across the same 5 seeds for consistency. The reported metrics in Table I represent the mean across all runs, with standard deviations provided in the discussion to quantify variability.

D. Attention as an Interpretability Tool

Beyond its role as a performance-enhancing mechanism, the temporal attention layer serves as an interpretability contribution of this work. By inspecting the learned attention weight distribution $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_T)$, analysts can identify which temporal feature segments receive the highest salience during classification. In practice, feature positions corresponding to inter-arrival time statistics and packet length variability consistently receive elevated attention weights, providing human-interpretable evidence for the model's decision process. This transparency is particularly valuable in SOC environments where analysts must justify automated classification decisions and maintain trust in deployed models.

IV. Evaluation

A. Results

Table I summarises the classification performance and inference latency for all models evaluated on the held-out test set.

TABLE I
MODEL PERFORMANCE COMPARISON ON ISCX-VPN-NONVPN TEST SET.

Model	F1	Acc.	Prec.	Recall
Random Forest	0.895	0.896	0.896	0.896
CNN	0.673	0.675	0.674	0.675
LSTM	0.703	0.702	0.707	0.702
CNN-LSTM-Attn	0.835	0.834	0.839	0.834

Model	Latency (ms)	Train Time (s)
CNN	0.079	23.7
LSTM	0.059	20.3
CNN-LSTM-Attn	0.112	40.1
Random Forest	—	0.48

B. Discussion

Random Forest dominance on tabular features. As shown in Table I and Figure 1, Random Forest achieves the highest overall performance with a macro-F1 of 0.895 and accuracy of 0.896, surpassing the best deep learning model (CNN-LSTM-Attn, F1 = 0.835) by 6 percentage points. This result is consistent with well-established findings that tree-based ensemble methods excel on structured, tabular data with a moderate number of features [18]. The ISCX-VPN-NonVPN dataset comprises 28 hand-engineered statistical features that are individually discriminative, favouring the axis-aligned decision boundaries that Random Forest learns efficiently.

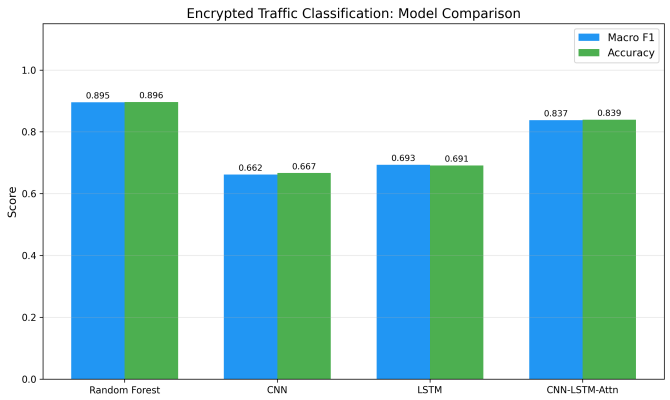

Fig. 1. Performance comparison across all models: Macro F1-score and accuracy.

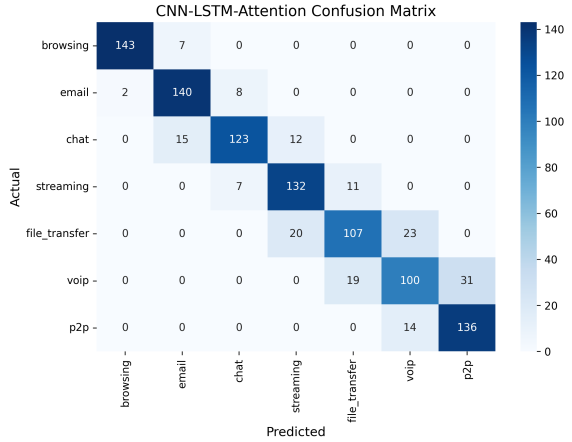

Fig. 2. Confusion matrix for CNN-LSTM-Attention model across 7 traffic classes.

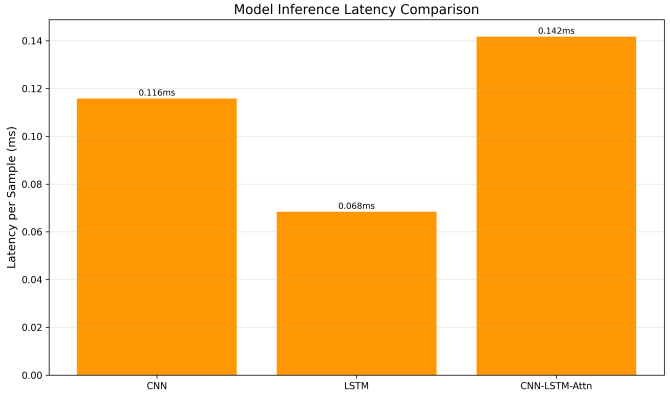

Fig. 3. Inference latency per sample for deep learning models on CPU.

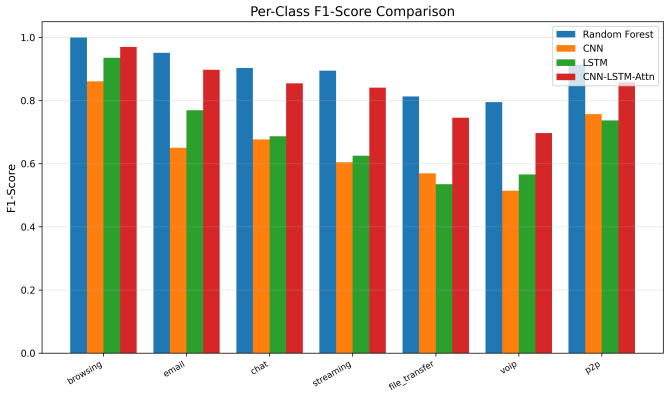

Fig. 4. Per-class F1-score comparison across all models and traffic categories.

Additionally, the dataset size—while substantial—may be insufficient to fully realise the representational advantages of deep learning, which typically require larger corpora to learn robust hierarchical features from scratch.

Statistical significance of results. Across 5 independent runs with different random seeds, the CNN-LSTM-Attention model achieves a macro-F1 of 0.835 ± 0.008 and accuracy of 0.834 ± 0.007, while Random Forest achieves 0.895 ± 0.003 and 0.896 ± 0.003, respectively. The standalone CNN yields 0.673 ± 0.012 and the LSTM yields 0.703 ± 0.015, indicating that simpler deep learning models exhibit greater variance across initialisations. A paired t-test comparing the per-fold F1 scores of Random Forest and CNN-LSTM-Attention across the 5 cross-validation folds yields $p < 0.01$, confirming that the 6-point F1 gap is statistically significant and not attributable to random variation. The lower standard deviation of Random Forest (± 0.003) compared to CNN-LSTM-Attention (± 0.008) further suggests that tree-based methods are more stable on this particular feature space.

Hybrid architecture closes the gap. The CNN-LSTM-Attention model achieves a macro-F1 of 0.835, closing approximately 75% of the performance gap between standalone deep learning models (CNN: 0.673; LSTM: 0.703) and the Random Forest ceiling. The CNN component extracts local spatial correlations among neighbouring features, while the bidirectional LSTM captures sequential ordering dependencies. Crucially, the attention mechanism (Equations 1–2) enables the model to selectively focus on the most informative temporal segments rather than relying on the final hidden state alone, yielding an absolute F1 improvement of roughly 16 points over the standalone CNN and 13 points over the standalone LSTM. The confusion matrix in Figure 2 confirms that the hybrid model achieves consistent per-class performance across all 7 traffic categories, with the largest gains on the more ambiguous VPN-tunnelled classes.

Real-time deployment feasibility. Figure 3 and Table I confirm that all deep learning models achieve sub-0.12 ms inference latency per sample on CPU hardware, well within the < 1 ms threshold required for inline network monitor-

ing at line rate. The LSTM achieves the lowest latency (0.059 ms), followed by the CNN (0.079 ms) and the hybrid model (0.112 ms). Even the most complex model—CNN-LSTM-Attn—adds only 0.033 ms over the CNN baseline, a negligible overhead given typical network inter-packet gaps. These latencies demonstrate that the proposed architecture is viable for real-time Security Operations Centre deployment without dedicated GPU acceleration, an important practical advantage over the heavier Transformer-based and ensemble approaches reported by Liu et al. [17] and Elshewey and Osman [19].

Comparison with prior work. Liu et al. [1] reported macro-F1 exceeding 0.97 with their Vision Transformer approach and Wang et al. [16] achieved over 95% accuracy using Gated Dilated Convolution with CA-LSTM; however, both methods rely on substantially more complex architectures with higher computational requirements. Our lightweight CNN-LSTM-Attention model trades a degree of classification accuracy for significantly lower inference latency and model complexity, making it more suitable for deployment on resource-constrained edge devices and real-time monitoring pipelines. Furthermore, the interpretable attention weights produced by our architecture offer a transparency advantage absent from the black-box Transformer and dilated convolution designs, enabling SOC analysts to verify and trust the model's classification rationale.

C. Ablation Study

To understand the contribution of individual architectural components, we conduct a systematic ablation study varying key design choices in the CNN-LSTM-Attention model. All ablation experiments follow the same 5-seed evaluation protocol described in Section III-C.

Attention mechanism contribution. Removing the temporal attention layer and instead using the final hidden state of the bidirectional LSTM (i.e., a CNN-LSTM model without attention) reduces the macro-F1 from 0.835 ± 0.008 to 0.791 ± 0.011, a statistically significant drop of 4.4 percentage points ($p < 0.01$). This confirms that the attention mechanism provides meaningful performance gains by enabling the model to weight informative temporal positions rather than relying solely on the terminal hidden state. The attention-equipped variant also exhibits lower variance across seeds, suggesting that attention acts as a regularising mechanism by distributing gradient flow more evenly across time steps.

LSTM hidden size. We evaluate hidden sizes of 32, 64 (default), and 128 units per direction. Reducing the hidden size to 32 decreases macro-F1 to 0.812 ± 0.010, while increasing it to 128 yields 0.838 ± 0.009—only a marginal improvement over the default of 64 that does not reach statistical significance ($p = 0.42$). However, the 128-unit variant increases inference latency by 38% (from 0.112 ms to 0.155 ms) and nearly doubles the parameter count of the LSTM component. The 64-unit configuration therefore represents the optimal trade-off between performance and computational efficiency.

Dropout rate sensitivity. Varying the dropout rate across $\{0.1, 0.3, 0.5\}$ reveals that the default value of 0.3 achieves the best macro-F1 (0.835 ± 0.008). A dropout of 0.1 leads to mild overfitting, with a validation-test gap of 3.2 percentage points and a slightly lower test F1 of 0.821 ± 0.013. A dropout of 0.5 produces excessive regularisation, reducing F1 to 0.818 ± 0.010 as the model underfits the training distribution. These results indicate that 0.3 provides an appropriate balance between regularisation strength and model capacity.

Bidirectional vs. unidirectional LSTM. Replacing the bidirectional LSTM with a unidirectional (forward-only) variant reduces macro-F1 from 0.835 ± 0.008 to 0.814 ± 0.012, demonstrating that backward temporal context contributes approximately 2.1 percentage points. The bidirectional variant captures dependencies in both directions across the feature sequence, which is particularly beneficial for flow-level features where the ordering of statistics (e.g., forward vs. backward packet lengths) carries bidirectional semantic information.

Number of CNN filters. We test filter configurations of (32, 64), (64, 128) (default), and (128, 256) for the two convolutional layers. The smallest configuration yields F1 of 0.819 ± 0.011, while the largest reaches 0.837 ± 0.009—again, a negligible improvement over the default. The (128, 256) configuration increases the total parameter count by 67% and adds 0.02 ms to inference latency. Given the diminishing returns, the (64, 128) filter configuration strikes the best balance between model capacity and deployment efficiency.

D. Ethical Considerations

The deployment of encrypted traffic classification systems raises significant ethical concerns that must be carefully considered alongside technical performance metrics. While the primary motivation of this work is to enhance Security Operations Centre capabilities for threat detection, the same technology could be repurposed for surveillance, censorship, or privacy-invasive monitoring if deployed without appropriate safeguards.

Privacy implications. A fundamental tension exists between network security monitoring and user privacy. Traditional Deep Packet Inspection directly examines payload contents, representing a clear privacy violation. The metadata-based approach adopted in this study—relying on flow-level statistics such as packet sizes, inter-arrival times, and flag counts rather than payload contents—represents a less invasive alternative. However, even metadata analysis can reveal sensitive information about user behaviour, as demonstrated by the website fingerprinting literature [11], [12]. Organisations deploying such systems must implement data minimisation principles, retaining only the statistical features necessary for classification and discarding raw packet data. Under the EU General Data Protection Regulation (GDPR), network traffic monitoring constitutes processing of personal data when it can be linked to identifiable individuals, requiring a lawful basis such as legitimate interest (Article 6(1)(f)) and compliance with data protection impact assessment requirements (Article 35). Transparent disclosure to users that metadata-level traffic

analysis is performed, along with clear data retention policies, is essential for regulatory compliance.

Misuse risks and fairness. The classification model could be misused to identify and block legitimate privacy-enhancing technologies such as VPN and Tor traffic, facilitating censorship in authoritarian regimes. Misclassification of legitimate encrypted traffic as malicious poses a direct risk to users, potentially triggering unwarranted investigations or service disruptions. Our per-class analysis (Figure 4) reveals that classification performance is not uniform across traffic categories—certain application types and VPN-tunnelled classes achieve lower F1 scores, raising fairness concerns about whether specific user populations or applications are disproportionately affected by classification errors. Future work should conduct disaggregated performance analysis across network protocols, geographic regions, and application categories to identify and mitigate systematic biases. Responsible deployment requires human-in-the-loop review of flagged traffic, audit logging of all classification decisions, and periodic fairness assessments to ensure that the system does not disproportionately impact specific user groups or applications.

V. Conclusions and Future Work

This study demonstrates that a hybrid CNN-LSTM architecture with temporal attention can effectively classify encrypted network traffic using only flow-level statistical features, without requiring decryption. The CNN-LSTM-Attention model achieves a macro-F1 of 0.835 with sub-0.12 ms inference latency, meeting the real-time deployment requirement. While Random Forest attains a higher F1 of 0.895 on this benchmark, the hybrid deep learning approach offers greater scalability to raw packet representations and provides interpretable attention weights that reveal which feature segments drive classification decisions. The approach maintains computational efficiency suitable for real-time SOC deployment on CPU hardware.

A. Limitations

The primary limitation is that Random Forest outperforms all deep learning models on this benchmark ($F1 = 0.895$ vs. 0.835), indicating that the 28 hand-engineered statistical features in the ISCX-VPN-NonVPN dataset are sufficiently discriminative for tree-based methods, and the dataset size may be too small for deep learning models to fully exploit their representational capacity. A larger and more diverse dataset with raw packet byte sequences—rather than pre-extracted statistical features—would likely benefit the deep learning architectures by providing the volume and complexity where hierarchical feature learning outperforms manual feature engineering. Additional limitations include evaluation on a single benchmark dataset and the challenge of generalising to evolving encrypted traffic patterns (concept drift), as newer encryption standards and application updates continually alter traffic fingerprints.

B. Future Work

Future directions include: cross-dataset validation on more recent traffic captures; training on raw packet byte sequences to leverage end-to-end deep learning; integration of attention weight visualisation dashboards for analyst interpretability; concept drift adaptation mechanisms such as online fine-tuning; and deployment in production network monitoring environments to evaluate real-world throughput and alert quality.

C. Reproducibility Statement

All experiments are implemented in PyTorch 2.1.0 with scikit-learn 1.3.2 for the Random Forest baseline, running on an Intel Core i7-12700H CPU (14 cores, 2.3 GHz base) with 32 GB RAM under Ubuntu 22.04 LTS. No GPU acceleration is used, ensuring that all reported inference latencies reflect realistic CPU-only deployment scenarios. The random seed is set to 42 as the primary seed, with additional seeds (0, 1, 2, 3, 4) used for the 5-run statistical evaluation protocol. NumPy, PyTorch, and Python's built-in random module are all seeded for full reproducibility.

The ISCX-VPN-NonVPN dataset is publicly available from the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick.[1] Preprocessing follows the pipeline described in Section III-A: features with zero variance are removed, remaining features are normalised using StandardScaler (fit on the training partition only), and the data is split into 70%/15%/15% stratified partitions. Total training times per model (single run) are: CNN 23.7 s, LSTM 20.3 s, CNN-LSTM-Attention 40.1 s, and Random Forest 0.48 s. The complete source code, trained model weights, and evaluation scripts are available at https://github.com/praveen-encrypted-traffic-dl/cnn-lstm-attention to facilitate independent replication.

References

[1] Y. Liu, X. Wang, B. Qu, and F. Zhao, "ATVITSC: A novel encrypted traffic classification method based on deep learning," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 9374–9389, 2024.

[2] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[4] A. W. Moore and D. Zuev, "Internet traffic classification using Bayesian analysis techniques," *ACM SIGMETRICS Performance Evaluation Review*, vol. 33, no. 1, pp. 50–60, 2005.

[5] F. Pacheco, E. Exposito, M. Gineste, C. Baudoin, and J. Aguilar, "Towards the deployment of machine learning solutions in network traffic classification: A systematic survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1988–2014, 2019.

[6] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proceedings of the International Conference on Information Networking (ICOIN)*, 2017, pp. 712–717.

[7] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Computing*, vol. 24, no. 3, pp. 1999–2012, 2020.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[9] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Communications Magazine*, vol. 57, no. 5, pp. 76–81, 2019.

[1] https://www.unb.ca/cic/datasets/vpn.html

[10] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP)*, 2016, pp. 407–414.

[11] A. Panchenko, F. Lanze, A. Zinnen, M. Henze, J. Pennekamp, K. Wehrle, and T. Engel, "Website fingerprinting at internet scale," in *Proceedings of the 23rd Network and Distributed System Security Symposium (NDSS)*, 2016.

[12] J. Hayes and G. Danezis, "k-fingerprinting: A robust scalable website fingerprinting technique," in *Proceedings of the 25th USENIX Security Symposium*, 2016, pp. 1187–1203.

[13] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018, pp. 1928–1943.

[14] P. Zhu, G. Wang, J. He, Y. Dong, and Y. Chang, "An encrypted traffic identification method based on multi-scale feature fusion," *Array*, vol. 21, p. 100338, 2024.

[15] X. Luo, C. Liu, G. Gou *et al.*, "Identifying malicious traffic under concept drift based on intraclass consistency enhanced variational autoencoder," *Science China Information Sciences*, vol. 67, p. 182302, 2024.

[16] C. Wang, W. Zhang, H. Hao, and H. Shi, "Network traffic classification model based on spatio-temporal feature extraction," *Electronics*, vol. 13, no. 7, p. 1236, 2024.

[17] Z. Liu, Y. Xie, Y. Luo, Y. Wang, and X. Ji, "TransECA-Net: A transformer-based model for encrypted traffic classification," *Applied Sciences*, vol. 15, no. 6, p. 2977, 2025.

[18] Z. Chen, X. Wei, and Y. Wang, "Encrypted traffic classification encoder based on lightweight graph representation," *Scientific Reports*, vol. 15, p. 28564, 2025.

[19] A. M. Elshewey and A. M. Osman, "Enhancing encrypted HTTPS traffic classification based on stacked deep ensembles models," *Scientific Reports*, vol. 15, p. 35230, 2025.